

Response to Reviewer Comments

Complex Engineering Problem (CEP): ORACLE

On-desk Responsive AI Companion with Local Environment

Project Team:

Faizan Ali (FA22-BCE-002)
 Shallum Safdar (FA22-BCE-008)
 Abdullah Laeeq (FA22-BCE-026)
 Aliyan Ahmed Cheema (FA22-BCE-028)

*Department of Computer Engineering
 COMSATS University Islamabad, Lahore Campus*

Dear Sir,

We would like to express our gratitude for your detailed review of our CEP topic, “ORACLE”. Your feedback regarding the alignment of our project with advanced VLSI design standards has been instrumental in elevating the academic rigor of our work. We have carefully reviewed your comments and have incorporated comprehensive VLSI modeling, CMOS design flows, and hardware-accelerated concepts into our revised report.

Below is a point-by-point response detailing how each of your recommendations has been addressed and integrated into our revised CEP submission.

Comment 1: *Extend the design beyond a microcontroller-based embedded system and incorporate a transistor-level CMOS VLSI design. You must implement critical modules (such as control logic, data handling unit, and interface controller) using CMOS logic, including proper PUN/PDN networks, logic-to-transistor mapping, and appropriate (W/L) sizing based on delay and load considerations.*

Response: We agree that relying solely on an off-the-shelf microcontroller limits the VLSI scope. In the revised report, we have designed a custom **Voice Activity Detection (VAD) & Control Module** at the transistor level. We mapped the critical control logic to CMOS architecture, explicitly detailing the Pull-Up Networks (PUN) and Pull-Down Networks (PDN). Furthermore, we have included mathematical justifications for our transistor sizing (W/L ratios), calculating the necessary widths to achieve symmetric rise and fall times while minimizing propagation delay under the specific capacitive loads of the I2S audio lines.

Comment 2: *Redesign the system architecture into a hardware-oriented VLSI model by clearly separating: Datapath, Control Unit (FSM), and Include a proper system-level block diagram and FSM state transition diagram.*

Response: The system architecture section has been completely overhauled to reflect a

true hardware-oriented VLSI model.

- **Datapath:** We have clearly isolated the datapath, detailing the audio data shift registers, I2S circular buffers, and 16-bit counters required for audio framing.
- **Control Unit (FSM):** We designed a strict hardware Finite State Machine governing the system. The transition logic covers the states: IDLE, LISTEN, PROCESS, DISPLAY, and AUDIO_OUTPUT.

A comprehensive system-level block diagram and a detailed state transition diagram have been added to Section 4 of the revised report to visualize this separation.

Comment 3: *Replace the purely software/API-driven pipeline with hardware-accelerated modules, such as: Voice trigger detection circuit (hardware-based), Data processing/control module in CMOS or FPGA, Interface controller for OLED/audio communication.*

Response: To reduce software dependency, we have modeled hardware-accelerated modules for the critical path. We introduced a hardware-based voice trigger detection circuit that utilizes an RMS-thresholding logic block to trigger the system wake state. Additionally, we modeled a custom I2S/I2C interface controller tailored specifically for our MEMS microphone and OLED display, illustrating how this data processing can be synthesized onto an FPGA or custom CMOS die to offload the central MCU.

Comment 4: *Complete the full VLSI design flow, including: RTL design and synthesis using Quartus, Functional simulation using ModelSim, CMOS schematic design (DSCH or equivalent), Layout design using Microwind (or similar tools), LVS (Layout vs Schematic verification), Post-layout simulation including parasitic analysis.*

Response: We have expanded our methodology to encompass the complete EDA flow for our custom I2S interface and VAD controller. The revised report now contains a dedicated section documenting:

- RTL code snippets and synthesis results via Intel Quartus.
- Timing waveforms and functional verification generated from ModelSim.
- Transistor-level schematics designed in DSCH.
- Physical layout representations generated using Microwind, utilizing 180nm technology nodes.
- Discussion on LVS verification and post-layout simulations, explicitly addressing the impact of parasitic capacitances on signal integrity.

Comment 5: *Add a detailed VLSI performance analysis section, including: Propagation delay and power equations, Identification of the critical path in your hardware design, Area estimation and layout considerations, Optimization techniques such as transistor sizing, buffering, and fan-out reduction.*

Response: A new “VLSI Performance Analysis” section has been added to the report. We provide mathematical models for static leakage and dynamic power dissipation ($P =$

$\alpha C_L V_{DD}^2 f$). We performed Static Timing Analysis (STA) to identify the critical path within the I2S buffer logic. To optimize this path, we documented the implementation of tapered buffering and optimal transistor sizing to mitigate fan-out delays, alongside area estimation based on our Microwind layout.

Comment 6: *Incorporate hardware-level innovation instead of only system integration, such as: Low-power design techniques, Hardware-efficient controller, Custom VLSI-based interface, Partial on-chip processing, Clearly justify novelty.*

Response: We have strengthened the novelty of the project by integrating explicit hardware-level innovations. We implemented clock gating techniques within the custom I2S controller to disable the clock tree during the IDLE state, drastically reducing dynamic power. The hardware VAD circuit serves as partial on-chip processing, ensuring the main processing cores remain in deep sleep until valid speech is detected. This hybrid approach (local edge-AI paired with custom low-power wake hardware) justifies the project’s novelty against standard, power-hungry commercial smart assistants.

Comment 7: *Ensure that your project is not purely API/cloud dependent, and introduce: Local hardware intelligence or preprocessing, Reduced reliance on external software systems.*

Response: This has been fully addressed in our recent architectural pivot. ORACLE is now entirely offline and independent of cloud APIs. The system relies on local hardware for audio preprocessing (hardware RMS thresholding), and leverages a local edge-server running `Ollama` (dolphin-phi) and `faster-whisper` for inference. The ESP32 handles direct hardware control (GPIO-based fan toggling) based on local intent parsing, demonstrating true local intelligence.

Comment 8: *Strengthen the implementation methodology by including: CMOS-level design diagrams, Detailed hardware architecture, Clear mapping from system design $\rightarrow$ RTL $\rightarrow$ transistor level $\rightarrow$ layout.*

Response: The implementation methodology has been completely restructured to follow a top-down VLSI design approach. We begin with the system-level block diagram, map the functional blocks to RTL, translate the RTL into CMOS gate-level schematics, and finally present the physical Microwind layouts. This step-by-step mapping clearly demonstrates our grasp of the complete VLSI lifecycle.

We believe these additions and hardware-level realizations comprehensively address your concerns and bring the ORACLE project up to advanced VLSI design standards. We submit this revised proposal for your approval.

Sincerely,

The ORACLE Project Team